Movie Night API

Laravel 13 + PostgreSQL · Information System Architecture · Midterm project plan

A Laravel rebuild of the Java Spring **Movie-Night** project: a clean-content movie catalog for family and friend movie nights. Each movie has content flags (nudity, violence, language) and optional “clean edit” info. Users can filter to family-safe movies, mark movies as watched, and review them.

This is the **midterm** version, so it stays close to what the course has covered. The bigger features are listed in section 3 and saved for the final project.

1. What the course has covered so far (weeks 1–4)

Week	Topic	Used in this project as
1	API-only Laravel 13, `routes/api.php`, JSON errors via `shouldRenderJsonWhen`	Same setup, no frontend
2	Controllers, Form Requests (`authorize` + `rules`), API Resources (`data` wrapper)	One Store/Update request and one Resource per entity
3	Migrations, Eloquent models with `#[Fillable]`, seeders, route model binding, `->whereNumber()`	All tables, models and seeders follow this style
4	`PATCH` partial updates with `sometimes`, `DELETE` with 204 No Content, status codes 200/201/204/404/422	Every update is `PATCH`, every delete returns 204

I'll write the code the way the teacher does: plain controllers (not `--api` resource routes), explicit `Route::get/post/patch/delete` lines with `whereNumber`, `#[Fillable([...])]` attributes on models, `$request->validated()` everywhere, validation like `in:none,mild,moderate,severe` (the same style as week 4's `in:open,closed`), and `firstOrNew` seeders that are safe to re-run.

2. What I add beyond the course (kept small for the midterm)

Only what the Movie Night idea needs to work:

1. **PostgreSQL** instead of SQLite, as you asked.
2. **Several connected tables.** The teacher's project has one table (tickets). Watches and reviews need foreign keys to users and movies, so I use the simple Eloquent relationships `hasMany` and `belongsTo`.
3. **Filters on** `GET /movies`: `?genre=` and `?clean=true`, the same as the Java version.
4. **The “family clean” rule**: each movie gets an `is_family_clean` field. A movie is family clean when every flag is `none` or `mild`, or when a clean edit is available. The Java version only checked nudity; this one checks all three flags.
5. **Average rating and review count** on each movie, like the Java version.
6. **Swagger docs with Scramble** (`dedoc/scramble`) at `/docs/api`. Scramble builds the docs automatically from the routes, Form Requests and Resources, so no extra code goes into the controllers. The teacher can try every endpoint from the browser. (The docs page needs internet to load.)
7. A **README** with setup steps and an endpoint table.

3. Saved for the final project

These are left out of the midterm on purpose:

- Login and tokens (Laravel Sanctum). For the midterm, requests send a `user_id` in the body, like the Java version did.
- Roles (admin / user) and authorization policies.
- PHP Enums for the content levels.
- Search, sorting and pagination.
- Extra review rules (review only after watching, one review per movie).
- Custom error responses, factories and automated tests.

4. Database design (PostgreSQL)

```
users                movies                          watches                   reviews
-----                ------                          -------                   -------
id                   id                              id                        id
username  (unique)   title                           user_id  FK -> users      user_id  FK -> users
email     (unique)   genre                           movie_id FK -> movies     movie_id FK -> movies
password  (hashed)   release_year                    watched_at                rating   (1-5)
timestamps           nudity_level     (none..severe) timestamps                comment  (nullable)
                     violence_level   (none..severe) UNIQUE(user_id,movie_id)  timestamps
                     language_level   (none..severe)
                     has_clean_edit   (true/false)
                     clean_edit_notes (nullable)
                     timestamps
```

- Deleting a movie or user also deletes its watches and reviews.
- IDs are integers to match the course (`whereNumber`). The Java version used UUIDs.

5. Endpoints (all prefixed with `/api`)

Users

Method	Path	Result
GET	`/users`	200, all users
GET	`/users/{user}`	200, user with watch history; 404 if not found
POST	`/users`	201, user created; 422 on invalid input

Movies

Method	Path	Result
GET	`/movies`	200, all movies; supports `?genre=` and `?clean=true`
GET	`/movies/{movie}`	200, flags, clean-edit info, average rating and reviews; 404 if not found
POST	`/movies`	201 / 422
PATCH	`/movies/{movie}`	200, partial update, including clean-edit fields / 422
DELETE	`/movies/{movie}`	204

Watching

Method	Path	Result
POST	`/movies/{movie}/watch`	body `{"user_id": 1}`; 201 first time, 200 if already watched

Reviews

Method	Path	Result
GET	`/movies/{movie}/reviews`	200, reviews for the movie
POST	`/movies/{movie}/reviews`	body `{"user_id": 1, "rating": 5, "comment": "..."}`; 201 / 422
PATCH	`/reviews/{review}`	200, change rating or comment / 422
DELETE	`/reviews/{review}`	204

6. Project structure

```
app/
  Http/Controllers/ UserController, MovieController, WatchController, ReviewController
  Http/Requests/    StoreUserRequest, StoreMovieRequest, UpdateMovieRequest,
                    WatchMovieRequest, StoreReviewRequest, UpdateReviewRequest
  Http/Resources/   UserResource, MovieResource, WatchResource, ReviewResource
  Models/           User, Movie, Watch, Review
database/
  migrations/       users, movies, watches, reviews
  seeders/          UserSeeder (3 demo users), MovieSeeder (~15 real movies), ReviewSeeder
routes/api.php
README.md
```

7. Build order

I'll build in phases, with one git commit per phase so the history reads like the teacher's weekly branches:

1. **Setup**: new Laravel 13 project in this folder, API-only (as in week 1), Postgres `.env`, `GET /api/hello` check.
2. **Movies**: migration, model, seeder, Resource, Store/Update requests, full CRUD (weeks 2–4 patterns).
3. **Filters**: `?genre=` and `?clean=true`, plus the `is_family_clean` field.

4. **Users**: migration, seeder, create, list and show.
5. **Watching and reviews**: migrations, relationships, endpoints, average rating on movies.
6. **Docs**: Scramble (Swagger) docs and README.
7. **Verification**: hit every endpoint against the real Postgres database and check the status codes.

8. What you need to do before I start

Your PC already has PHP 8.4, Composer and PostgreSQL 18. Two things are left:

1. **Turn on the Postgres driver in PHP.** Open `C:\Users\home\AppData\Local\Microsoft\WinGet\Packages\PHP.PHP.8.4_Microsoft.Winget.Source_8wekyb3d8bbwe\php.ini` and remove the `;` in front of `extension=pdo_pgsql` and `extension=pgsql`. I can make this edit for you if you prefer.
2. **Your Postgres password.** I won't type your password anywhere. After I create `.env`, you put it in the `DB_PASSWORD=` line yourself. Then I create the `movie_night` database (or you can create it in pgAdmin).

9. Defaults

Question	My default
Project location	Directly in `C:\Users\home\Downloads\RestAPI_midterm_project`
Git	`git init` with one commit per phase. Nothing gets pushed to GitHub unless you ask.